



MLP 可以处理变长序列吗？ Why?

前馈神经网络通常接受固定大小的输入，并产生固定大小的输出。这种固定的结构使得它们难以处理具有不同长度的序列数据。



CNN擅长处理
变长序列吗？
Why?



假设你正在英语考试…



六级单词背完了

掌握了英语语法的规律

掌握了句子结构

做过一些试卷

结合语境可以推出陌生单词的含义

假设你正在英语考试...



今天天气真好！

The weather is really nice today!

Schools should also provide access to libraries during lessons throughout the years of schooling. Girls visit the library in their free time more than boys, and as students move to higher grades they often have less access to libraries during class. Access to books for boys is essential to promote reading.

In addition, we should keep paper books available because research shows that boys are less likely to choose to read on screens than girls. Finally we should promote reading as an enjoyable pastime by being a role model. Let your children or students see you read for pleasure.

24. According to the author, what is the main cause of the difference in boys' and girls' reading ability?

- A. Social factors.
- B. Teaching methods.
- C. Personal preferences.
- D. Psychological differences.

假设你正在英语考试….

六级单词背完了

掌握了英语语法的规律

掌握了句子结构

做过一些试卷

结合语境可以推出陌生单词的含义



所以一个英语拿高分
的学生需要有哪些能力？

经验是智慧之父,记忆是智慧之母.

——谚语

提高它们处理**序列数据**、捕获**长期依赖**、增强**推理能力**、处理复杂关系、提高模型性能、**适应动态环境**以及更接近于人类智能水平的能力。这些好处使得具有记忆能力的神经网络在许多应用中变得更加有吸引力和有效。



为什么要给神经网络增加记忆能力？

循环神经网络 (RNN)：RNN是一种具有记忆功能的神经网络，它能够通过隐藏层的状态来保存和传递信息，从而实现对序列数据的记忆和处理。RNN在处理时间序列数据和自然语言处理任务中表现出色。

引入**外部记忆单元**：在神经网络中引入外部记忆单元，如长短期记忆 (LSTM) 或门控循环单元 (GRU)，可以扩展神经网络的记忆能力。这些外部记忆单元可以存储和检索历史信息，从而帮助神经网络更好地理解 and 处理序列数据。

采用**注意力机制**：注意力机制可以帮助神经网络关注重要的输入信息，并忽略不相关的信息。通过引入注意力机制，神经网络可以更加关注与当前任务相关的信息，从而提高其记忆和推理能力。

等等

怎么给神经网络增加记忆能力？



第八章 循环神经网络与 自然语言处理

本课程目标



了解循环神经网络的概念和特性



掌握使用RNN进行自然语言处理的基本过程



了解RNN的训练过程



了解LSTM的概念和特性



了解GRU的概念和特性

课程目录

Course catalogue

1/ 门控记忆机制

2/ 循环神经网络应用于自然语言处理

- RNN
- LSTM
- GRU

序列建模的 门控记忆机制

RNN · LSTM · GRU 数学原理

动机 · 架构 · 完整推导

为什么需要循环神经网络？

核心问题

序列数据具有**时序依赖**——当前时刻的语义依赖于历史

前馈网络要求固定大小输入，无法天然处理：

- 变长文本、语音、时间序列
- 跨时刻的状态传递
- 参数随序列长度的共享

核心思想

$$h_t = f(h_{t-1}, x_t)$$

用同一个函数 f 反复作用于序列，把历史压缩进隐藏状态 h_t 。
参数在所有时间步共享 → 模型规模与序列长度解耦。

Simple RNN

最朴素的循环
梯度消失/爆炸

LSTM

门控 + cell state
长程记忆能力强

GRU

简化门控
参数更少、训练更快

Simple RNN: 把循环写成参数共享

设计动机

前馈网络无法表达跨时刻依赖。最直接的做法是：

- ① 引入一个状态向量 h_t 作为「历史摘要」
- ② 每一时间步，用同一组参数把 h_{t-1} 与新输入 x_t 融合
- ③ 输出可以从 h_t 中线性读出

这就是 Elman 1990 提出的 Simple RNN 雏形。

递推公式

$$h_t = \tanh (W_x x_t + W_h h_{t-1} + b)$$

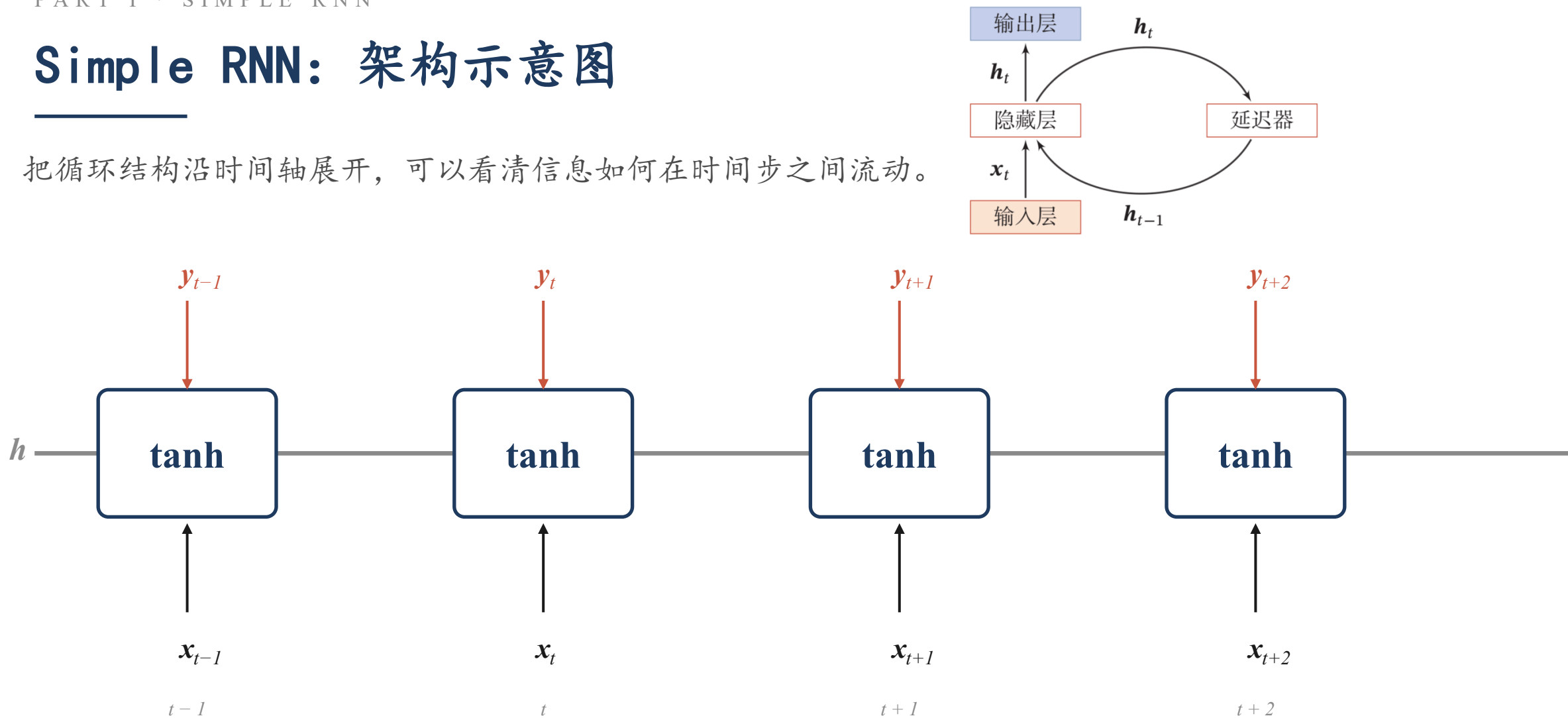
$$y_t = W_y h_t + b_y$$

关键性质

- 参数 (W_x , W_h , b) 在所有时间步共享
- 时间维度上展开 = 一个很深的前馈网络
- 训练用 BPTT (Backpropagation Through Time)

Simple RNN: 架构示意图

把循环结构沿时间轴展开，可以看清信息如何在时间步之间流动。



每个 $\tanh$ 单元的参数 (W_x, W_h, b) 完全共享——只有这一组参数，被反复使用 T 次。

Simple RNN: 梯度消失与爆炸

通过时间的反向传播 (BPTT)

设损失为 L ，关心初始状态 h_0 对 t 时刻 loss 的梯度：

$$\partial h_t / \partial h_0 = \prod_{k=1..t} \text{diag}(\tanh') \cdot W_h$$

两个连乘因子决定梯度的命运：

- ① $\tanh'(\cdot) \in (0, 1]$
绝大多数情况严格 $< 1 \rightarrow$ 连乘指数衰减
- ② W_h
特征值 $< 1 \rightarrow$ 梯度消失；特征值 $> 1 \rightarrow$ 梯度爆炸

数值直觉

每步因子	t = 10	t = 50	t = 100
0.9	0.35	0.005	≈ 0
0.5	0.001	≈ 0	≈ 0
1.1	2.6	117	13780

瓶颈

Simple RNN 的梯度被 同一个矩阵 W_h 反复相乘，衰减率不可学习、不可控。

$\rightarrow$ 难以学到超过 ~ 10 步的长程依赖。

这正是 LSTM 要解决的问题。

LSTM：把梯度路径从乘性改成加性

设计动机

Hochreiter & Schmidhuber (1997) 的核心洞察：

梯度消失的根源是 h_t 沿时间反向传播时被**矩阵反复相乘**（乘性更新）。

如果开辟一条**加性**的「记忆高速公路」，
梯度就可以近乎无损地传过去。

实现方式：

- 引入独立的 **cell state** c_t 作为长期记忆
- 用**门控** (gate) 来决定保留 / 写入 / 输出多少
- cell state 用**加法**而非矩阵乘法更新

乘性 vs 加性 更新

Simple RNN (乘性)

$$h_t = \tanh(W \cdot h_{t-1} + \dots)$$

梯度沿 W 连乘 → 指数衰减/爆炸

LSTM (加性)

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

梯度沿 cell state 加性传递 → 可控保留

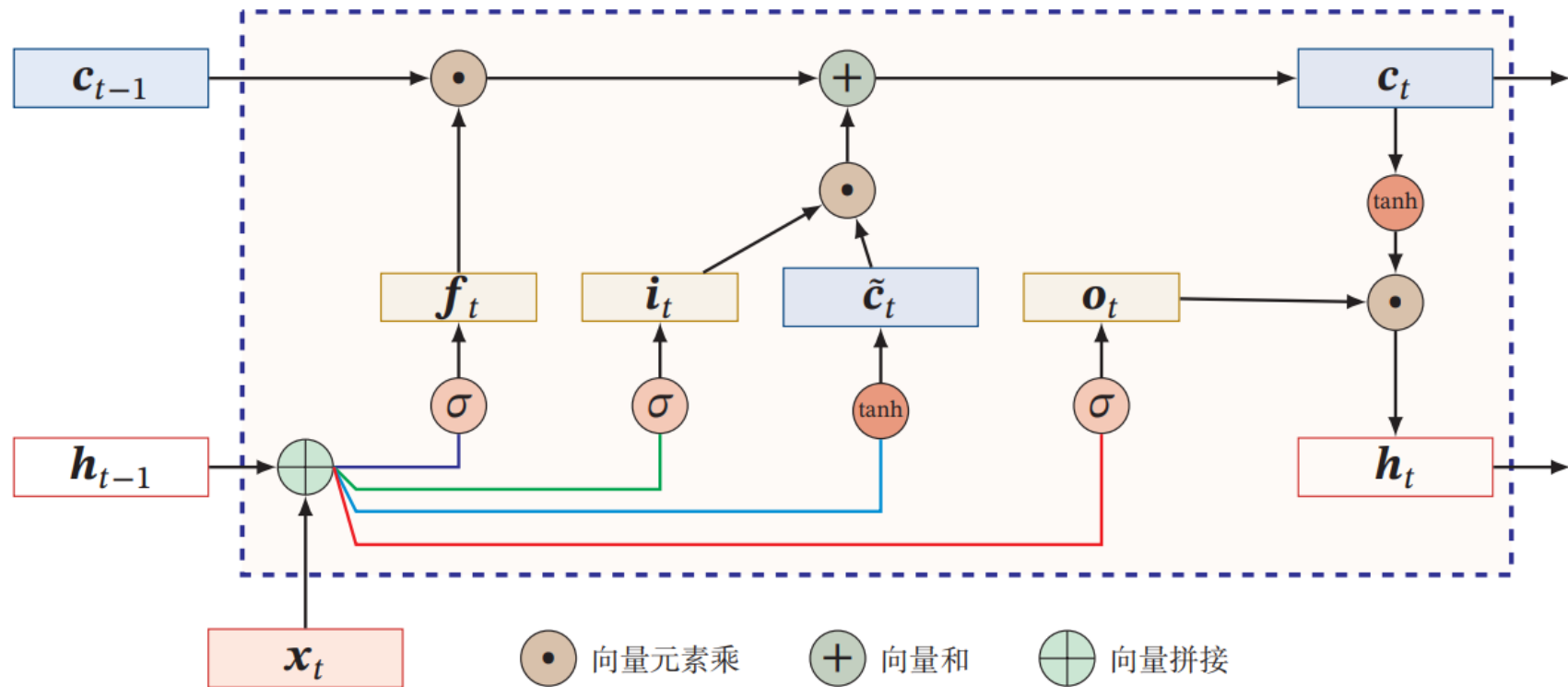
「Constant Error Carousel」——Hochreiter 用来描述 LSTM 让误差沿 cell state 不衰减地循环回传

LSTM: 单元结构示意图

$$\begin{bmatrix} \tilde{\mathbf{c}}_t \\ \mathbf{o}_t \\ \mathbf{i}_t \\ \mathbf{f}_t \end{bmatrix} = \begin{bmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{bmatrix} \left(\mathbf{W} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{h}_{t-1} \end{bmatrix} + \mathbf{b} \right),$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$



LSTM: 完整方程

给定输入 x_t 与前一步状态 (h_{t-1}, c_{t-1}) ，按下列六个公式完成一个时间步：

遗忘门

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

输入门

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

候选记忆

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

更新 cell state

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

输出门

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

hidden state

$$h_t = o_t \odot \tanh(c_t)$$

记号约定： σ = sigmoid, $\odot$ = 逐元素乘 (Hadamard product); $W_* \in \mathbb{R}^{d \times n}$, $U_* \in \mathbb{R}^{d \times d}$; 总参数 $4d(n+d+1)$

LSTM 为什么能缓解梯度消失？

沿 cell state 反向传播

对更新式 $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ 求偏导：

$$\partial c_t / \partial c_{t-1} \approx f_t$$

于是长程梯度沿 cell state 累积为：

$$\partial c_T / \partial c_0 \approx \prod_{k=1..T} f_k$$

关键差别：

Simple RNN 的连乘因子是固定的 $W \cdot \tanh'$ ；

LSTM 的连乘因子是**可学习的 f_k** ——模型可以选择让它接近 1。

可学习的衰减率

遗忘门 f	$t = 100$	$t = 1000$
0.90	2.7×10^{-5}	≈ 0
0.99	0.37	4.3×10^{-5}
0.999	0.90	0.37

门控的表达瓶颈

Sigmoid 让 $f_t \in (0, 1)$ 严格小于 1，
cell state 的传递依赖 $\prod f_k$ 的连乘——
即使 $f_k \approx 1$ ，梯度仍指数衰减。

→ **实际记忆长度仍存在上限。**

实践 trick：把 b_f 初始化为 1，让 f_t 训练初期偏向「记住」

GRU: 能不能更简单?

设计动机

LSTM 强大，但代价是：

- 3 个门 + 1 个候选 = 4 组参数
- cell state 和 hidden state 两条记忆线

Cho et al. (2014) 提出 GRU 的核心问题：

- ① 真的需要 cell 和 hidden 两条线吗？
- ② 输入门和遗忘门能不能合并？
- ③ 输出门是必要的吗？

GRU 的回答：

- 合并 c 和 h，只留 一条 hidden state
- 写入与遗忘互补：用 1 个更新门 z_t 同时控制两者
- 增加一个重置门 r_t ，控制历史对候选值的影响
- 去掉输出门

结构对照

LSTM

门: f, i, o

状态: c_t (cell) + h_t (hidden)

参数: $4d(n + d + 1)$

GRU

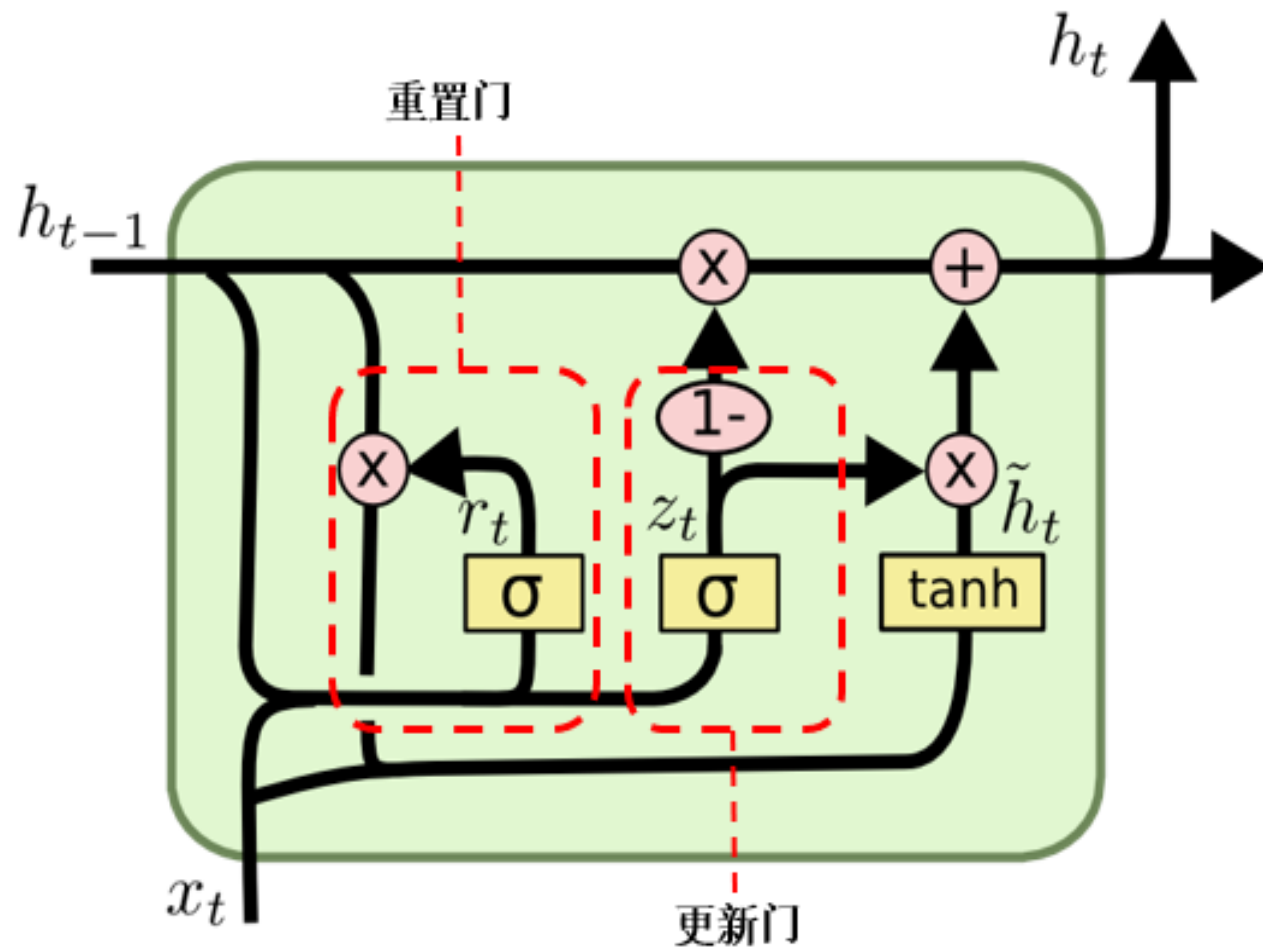
门: z, r

状态: 仅 h_t

参数: $3d(n + d + 1)$ ($\approx 75\%$ of LSTM)

GRU: 单元结构示意图

两个门 + 一条 hidden state——比 LSTM 更紧凑，没有独立的 cell state。



GRU: 完整方程

给定 x_t 与前一步 h_{t-1} ，一个时间步只需四个公式：

重置门

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

更新门

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

候选状态

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

更新 hidden

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

两个直觉

- ① 更新门 z_t 是 LSTM 中 f 和 i 的合并体： $(1 - z_t)$ 保留旧的， z_t 写入新的——二者强制互补。
- ② 重置门 r_t 决定候选状态「向过去看多远」： $r_t \approx 0$ 几乎只看当前输入； $r_t \approx 1$ 充分利用历史。

三个关键 takeaways

01

循环 = 参数共享

RNN 的本质是把同一组参数沿时间反复使用，让模型规模与序列长度解耦。这是所有循环架构的共同地基。

02

门控 = 梯度路径设计

LSTM 的真正贡献不是「记忆单元」，而是把信息流从乘性矩阵改成加性 cell state，让梯度衰减率变成可学习参数。

03

简化 = 务实折中

GRU 证明：合并互补的门、去掉冗余的状态，可以用 $\frac{3}{4}$ 的参数逼近 LSTM 的效果。这是工程审美胜过理论完整性的典型案例。

延伸阅读

Hochreiter & Schmidhuber (1997) · Cho et al. (2014) · Jozefowicz et al. (2015) *An Empirical Exploration of RNN Architectures* · Vaswani et al. (2017) *Attention Is All You Need*

课程目录

Course catalogue

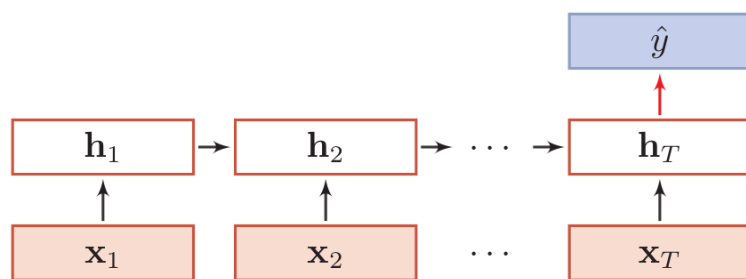
1/ 门控记忆机制

2/ 循环神经网络应用于自然语言处理

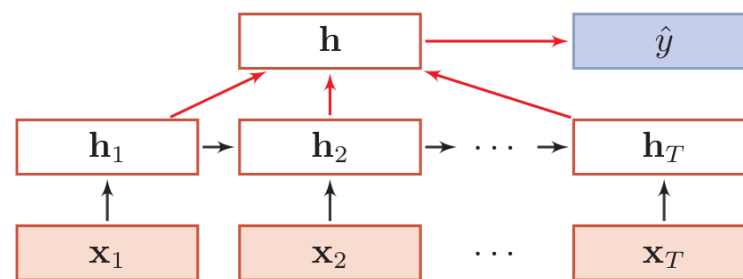
循环神经网络应用于自然语言处理

序列到类别

序列到类别模式是指对整个序列进行分类，该类问题包括情感分析、主题分类、垃圾邮件检测等。在所有的这些应用中，输入为文本序列，输出为几种类别之一。



(a) 正常模式

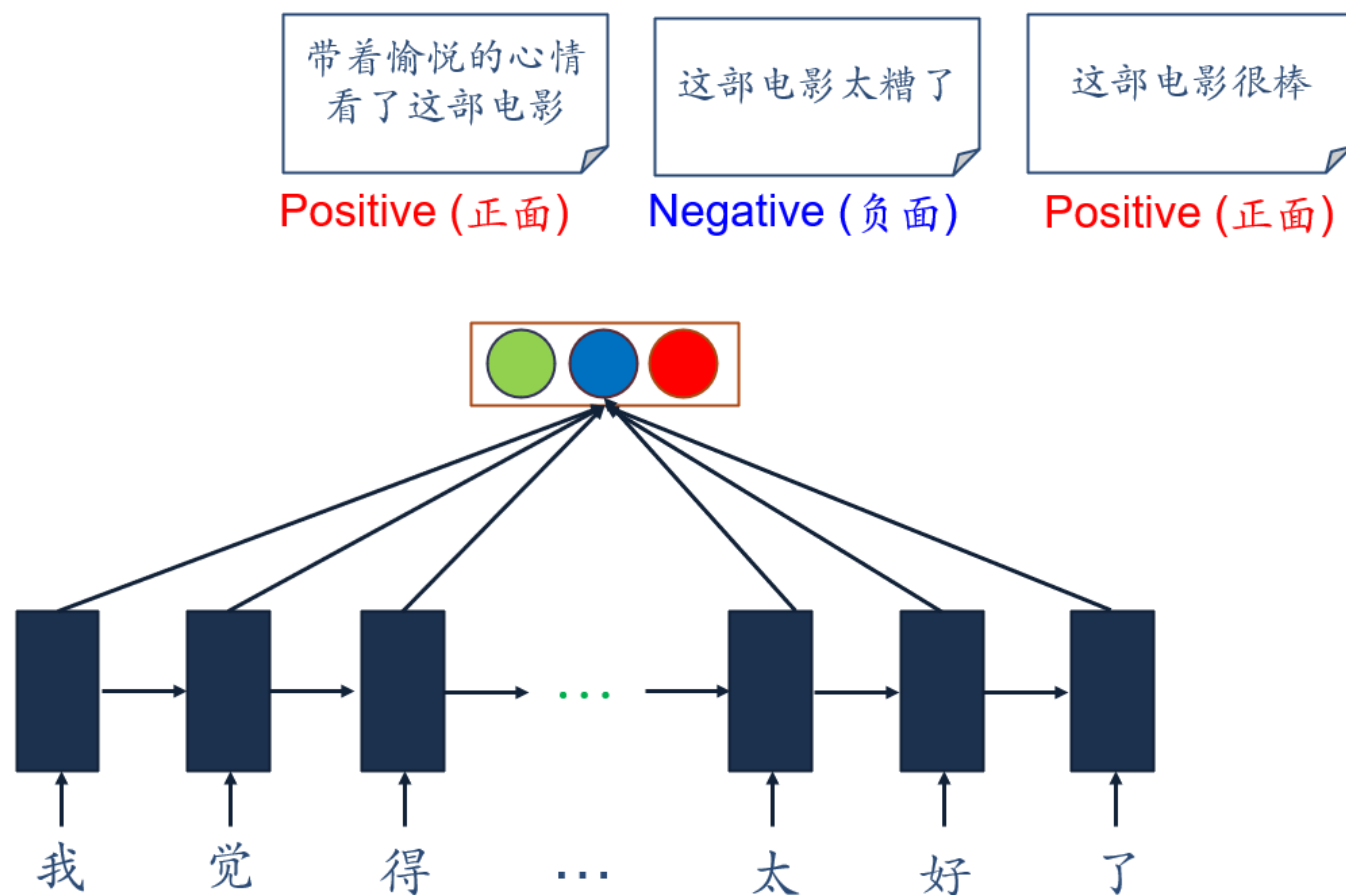


(b) 按时间进行平均采样模式

循环神经网络应用于自然语言处理

序列到类别

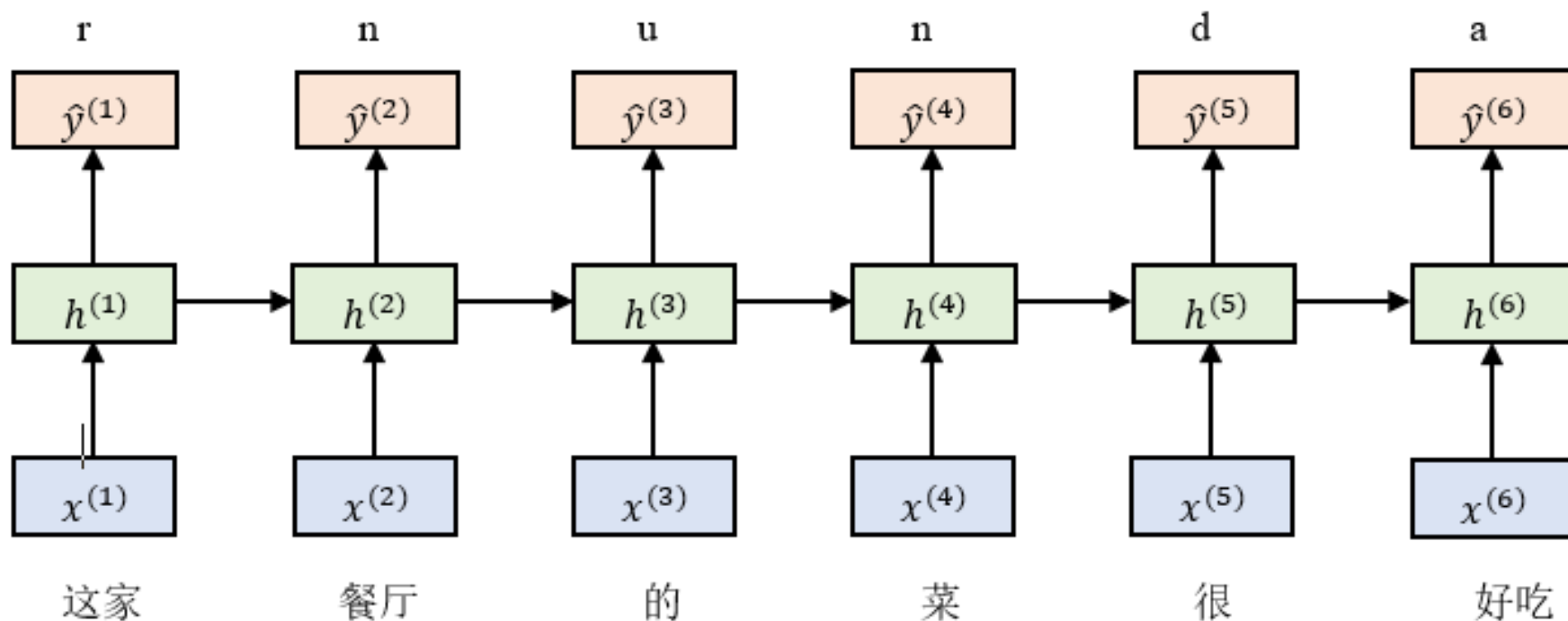
►情感分类



循环神经网络应用于自然语言处理

同步序列到序列

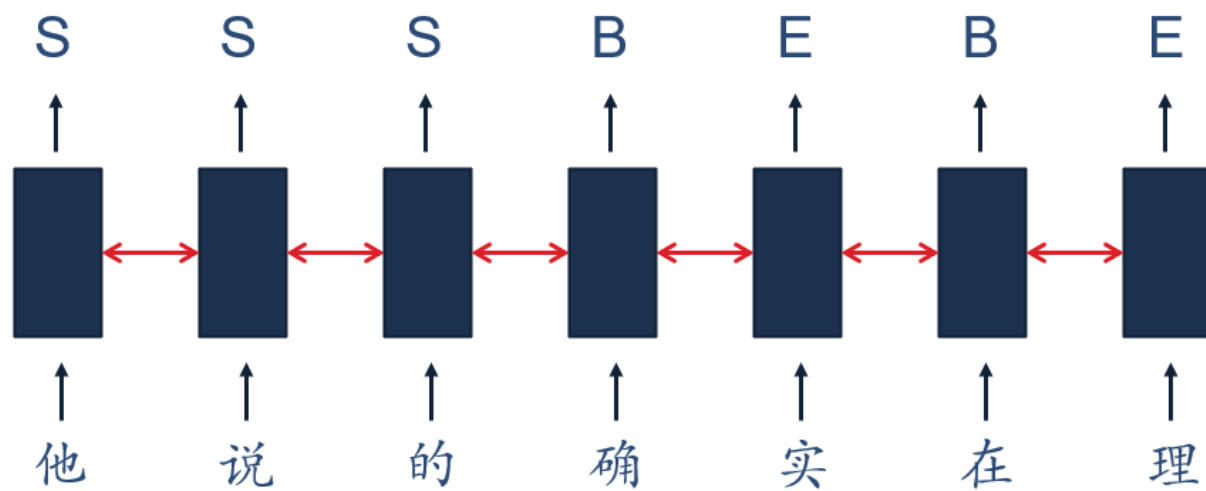
同步序列到序列的模式主要用在序列标注（Sequence Labeling）任务中，网络需要在固定的小型标签集中选择相应的标签分配给序列的每个元素，因此，每个时间步都有输入和输出，且输入序列与输出序列长度相同。



循环神经网络应用于自然语言处理

同步序列到序列

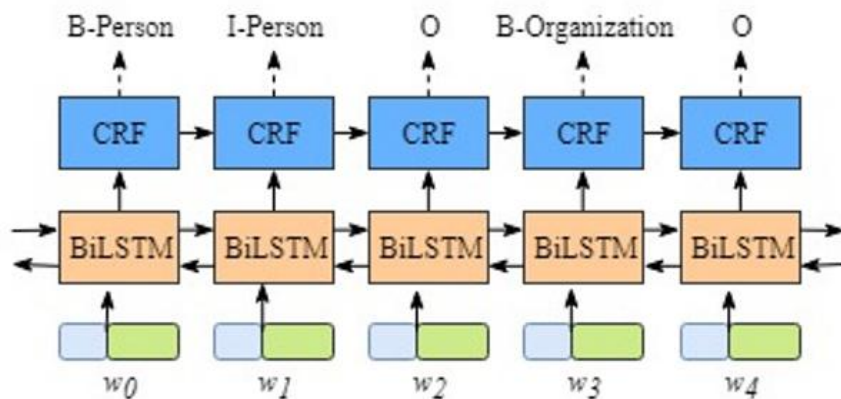
► 中文分词



同步序列到序列

- ▶ 信息抽取(Information Extraction, IE)
- ▶ 从无结构的文本中抽取结构化的信息，形成知识

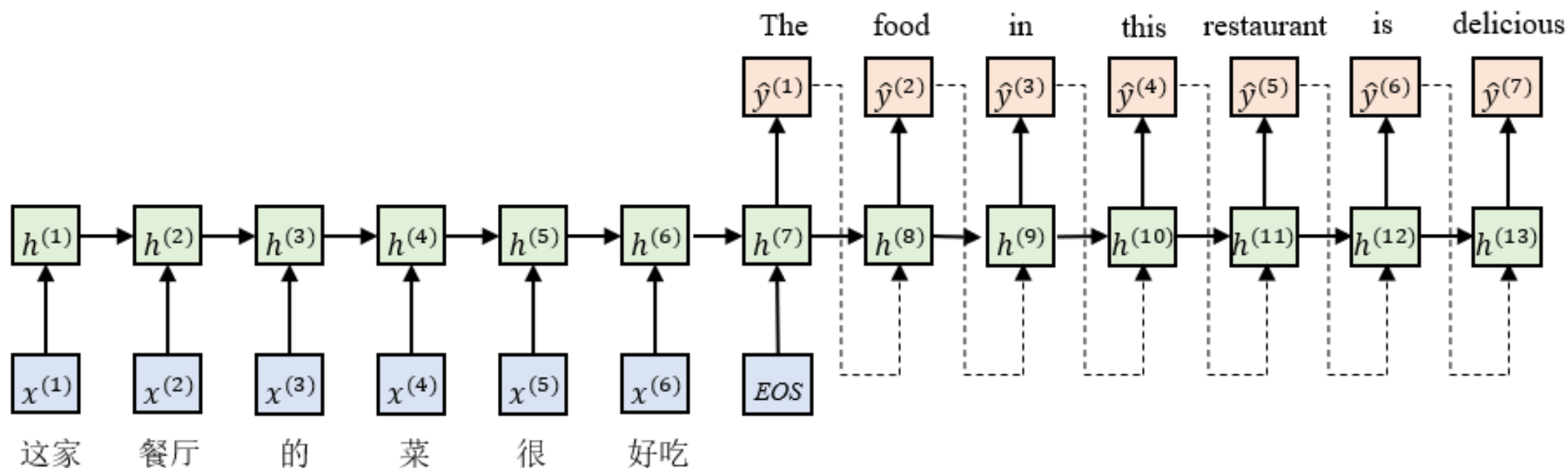
小米创始人雷军表示，该公司2015年营收达到780亿元人民币，较2014年的743亿元人民币增长了5%。



循环神经网络应用于自然语言处理

异步序列到序列

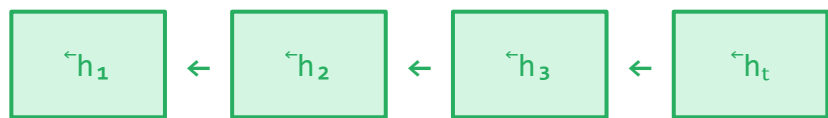
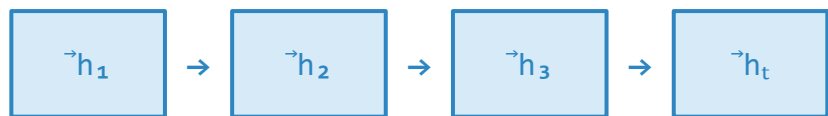
异步序列到序列的模式指输入序列和输出序列不必有严格的一一对应关系，也不需要具有相同的长度，也称为编码器-解码器（Encoder-Decoder）模型。此类模型的代表性应用包括机器翻译、对话系统、自动摘要等。



前沿科研补充

双向LSTM (BiLSTM)

核心思想



x_1

x_2

x_3

x_t

单向RNN只能利用
过去的上下文信息

BiLSTM同时运行两个LSTM:

前向LSTM: $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t$

后向LSTM: $x_t \rightarrow \dots \rightarrow x_2 \rightarrow x_1$

最终隐状态拼接:

$$h_t = [\vec{h}_t ; \overleftarrow{h}_t]$$

经典应用: 命名实体识别 (NER)、关系抽取、机器阅读理解、语义角色标注

代表论文: *Graves & Schmidhuber (2005) Framewise Phoneme Classification with BiLSTM*

Speech Recognition with Deep Recurrent Neural Networks

Graves, Mohamed & Hinton · ICASSP 2013 | 深层循环神经网络的奠基之作

论文基本信息

"Speech Recognition with Deep Recurrent Neural Networks"

Alex Graves, Abdel-rahman Mohamed, Geoffrey Hinton

ICASSP 2013 · 引用量 3000+ · 任务: TIMIT 基准语音识别

Deep LSTM

CTC Loss

SOTA 2013

研究背景 · 原始RNN的痛点

⚠ 已有方法的局限

- ⚠ 单层RNN 表达能力弱, 难以建模复杂时序层次
- ⚠ 梯度消失导致长程依赖遗忘
- ⚠ 传统语音模型 (HMM+GMM) 需要大量手工特征工程
- ⚠ 深层前馈网络忽略时序上下文连续性

核心贡献

✓ 三大创新

- 1 多层堆叠LSTM: 首次系统验证深层RNN有效性, 低层学局部模式, 高层学抽象规律
- 2 双向深层RNN (DBRNN): 每层同时利用前向+后向上下文, 大幅提升特征质量
- 3 端到端训练 (CTC): 无需帧级对齐标注, 直接优化序列级目标

实验结果 (TIMIT 数据集)

📊 Phone Error Rate (PER) ↓ 越低越好

23.9%

Deep BRNN (5层)

17.7%

Deep LSTM + CTC

17.1%

🏆 SOTA (当时)

Deep LSTM

Graves et al. (2013)

Deep LSTM 架构示意（5层堆叠）



关键洞察

- 为什么堆叠有效？
 - 层次化表征：低层捕捉帧级声学细节，高层提取音素/词级抽象模式
 - 梯度传播更稳：LSTM 门控机制使深层梯度不易消失
 - 类比 Deep CNN：卷积层越深，特征越抽象；LSTM 同理

实践设计原则

- 工程经验
 - 2—4 层通常最佳，过深收益递减且训练不稳定
 - 层间加 Dropout（0.2—0.5）防止过拟合
 - 可引入 Residual Connection 缓解深层退化

💡 TAKE-AWAY · 对本课程的启示

深层 LSTM 并非简单堆叠——每层学习不同时间尺度的特征表示，这一思想直接启发了 Seq2Seq、Attention 乃至 Transformer 的层次化设计。时序预测中合理选择层数，往往比调超参更重要。

注意力机制与LSTM

为什么需要Attention?

传统Seq2Seq的瓶颈:

编码器将整个输入序列压缩成一个固定长度的向量 c ,

当序列较长时信息严重丢失。

Attention的解决思路:

解码每个词时, 动态「回看」编码器所有时间步的

隐藏状态, 加权求和得到上下文向量:

$$\alpha_i = \text{softmax}(\text{score}(s_t, h_i))$$

$$c_t = \sum \alpha_i \cdot h_i$$

三种打分函数

Dot-Product

$$\text{score} = s_t^T h_i \quad (\text{最简单、无参数})$$

Bilinear (Luong)

$$\text{score} = s_t^T W h_i \quad (\text{可学习的} W \text{矩阵})$$

Additive (Bahdanau)

$$\text{score} = v^T \tanh(W_1 s_t + W_2 h_i)$$

RNN vs Transformer：现代NLP中的定位

维度	RNN/LSTM/GRU	Transformer
序列建模	天然按时序处理， 隐式捕获顺序	通过位置编码 显式注入位置信息
并行能力	逐步计算，难以并行 训练速度慢	完全并行 训练效率高
长距离依赖	理论上可捕获， 实际仍受梯度制约	自注意力直接连接 任意距离位置
参数效率	参数共享，模型紧凑 适合小数据/边缘部署	参数量大 需要大规模数据
可解释性	隐状态难以直接 解释	注意力权重提供 一定可解释性

关键结论：Transformer在大规模预训练中占主导，但RNN在低资源、实时推理、边缘部署等场景仍有不可替代的优势。

RNN的「现代复兴」：从xLSTM到Mamba

xLSTM (Beck et al., 2024)

Sepp Hochreiter团队（LSTM发明者）提出

- sLSTM: 引入指数门控，标量记忆
- mLSTM: 矩阵记忆，类似线性注意力
- 在语言建模上接近Transformer性能

Mamba / S4 (Gu et al., 2023-2024)

状态空间模型 (SSM) 系列

- 选择性状态空间：输入依赖的门控
- 线性时间复杂度，硬件友好
- Mamba-2在多项任务超越Transformer

RWKV (Peng et al., 2023)

结合RNN推理效率与Transformer训练并行性

- 线性复杂度的注意力近似
- 14B参数模型，可与GPT-3媲美
- 开源社区活跃，支持中文

为什么「复兴」？

Transformer的二次复杂度在长序列上瓶颈明显

- 128K+ tokens场景：RNN变体更高效
- 边缘设备：线性内存消耗是关键优势
- 实时流式处理：RNN天然适配

趋势：现代架构正在融合RNN的顺序处理优势与Transformer的并行训练能力，形成「混合架构」新范式。

推荐阅读: Beck et al. (2024) "xLSTM: Extended Long Short-Term Memory" | Gu & Dao (2024) "Mamba: Linear-Time Sequence Modeling" | Peng et al. (2023) "RWKV"

RNN/LSTM在NLP中的经典论文路线图



现代延续 (2018-2024): ELMo (Peters 2018) → ULMFiT (Howard 2018) → xLSTM (Beck 2024) → Mamba (Gu 2024)

从LSTM到Attention再到Transformer, RNN系列的每一步演进都深刻影响了NLP的发展方向。

PyTorch 代码实践指南

RNN / LSTM / GRU — 从零搭建到实战

PyTorch三大循环层 API 对比

```
# 基础RNN
rnn = nn.RNN(
    input_size=300,    # 词向量维度
    hidden_size=128,   # 隐藏层大小
    num_layers=2,      # 堆叠层数
    batch_first=True,  # (batch, seq, feat)
    bidirectional=True # 双向
)
out, h_n = rnn(x)  # x: (B, T, 300)
```

```
# LSTM (额外输出细胞状态)
lstm = nn.LSTM(
    input_size=300,
    hidden_size=128,
    num_layers=2,
    batch_first=True,
    dropout=0.3,      # 层间dropout
)
out, (h_n, c_n) = lstm(x)
```

```
# GRU (更简洁的门控)
gru = nn.GRU(
    input_size=300,
    hidden_size=128,
    num_layers=2,
    batch_first=True,
    bidirectional=True
)
out, h_n = gru(x)
```

关键区别			
特征	RNN	LSTM	GRU
门控数量	0	3 (输入/遗忘/输出)	2 (重置/更新)
隐藏输出	h_n	(h_n, c_n)	h_n
参数量	最少	最多 (≈ 4x RNN)	中等 (≈ 3x RNN)
推荐场景	简单序列	长序列/复杂任务	资源有限/快速实验

PyTorch实战：LSTM情感分类模型

```
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
                             num_layers=2, batch_first=True,
                             dropout=0.3, bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, n_classes) # *2因为双向
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # x: (batch, seq_len)
        emb = self.dropout(self.embedding(x)) # (B, T, embed_dim)
        out, (h_n, _) = self.lstm(emb) # h_n: (4, B, H)
        # 拼接最后一层的前向和后向
        h_cat = torch.cat([h_n[-2], h_n[-1]], dim=1) # (B, 2H)
        return self.fc(self.dropout(h_cat)) # (B, n_classes)
```

核心要点

① Embedding层

将词索引映射为密集向量，
可加载预训练Word2Vec/GloVe

② BiLSTM层

双向读取上下文，
2层堆叠提取更高层特征

③ 取最终隐藏状态

$h_n[-2]$ 前向最后层 +
 $h_n[-1]$ 后向最后层

④ Dropout正则化

嵌入层后 + 分类层前
防止过拟合

数据集推荐：IMDB (50K 影评) | SST-2 (Stanford) | 中文: ChnSentiCorp (“谷歌店铺评论”) | Weibo100K

训练Pipeline与调参建议

```
# 训练循环核心代码
model = SentimentLSTM(vocab_size=30000,
                      embed_dim=300, hidden_dim=256, n_classes=2)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for epoch in range(20):
    model.train()
    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()
        logits = model(batch_x)
        loss = criterion(logits, batch_y)
        loss.backward()
        # ★ 梯度裁剪 (防止梯度爆炸)
        nn.utils.clip_grad_norm_(
            model.parameters(), max_norm=5.0)
        optimizer.step()
```

实践Tips

梯度裁剪

RNN必备! `max_norm=5.0`是常用起点。
没有它, 训练很可能爆炸/NaN

序列Padding

`pack_padded_sequence` 可避免
LSTM在PAD上浪费计算

学习率调度

`ReduceLROnPlateau` 或 `CosineAnnealing`
通常从`lr=1e-3`开始

预训练嵌入

加载GloVe/Word2Vec可提升2-5%
冻结几个epoch后再解冻微调

Early Stopping

监控`val loss`, 连续5个epoch不降
则停止训练, 防止过拟合

完整代码参考: github.com/bentrevett/pytorch-sentiment-analysis (BiLSTM情感分类教程)

实用技巧：变长序列处理与 pack_padded_sequence

```
from torch.nn.utils.rnn import (
    pack_padded_sequence, pad_packed_sequence)

# 1. 假设 batch 中句子长度不同
lengths = [15, 10, 8, 3] # 已按长度降序
x_padded = pad_to_max_len(batch) # (4, 15, 300)

# 2. 压缩: 跳过PAD位置的计算
packed = pack_padded_sequence(
    x_padded, lengths,
    batch_first=True,
    enforce_sorted=False # 允许未排序
)

# 3. 送入LSTM
packed_out, (h_n, c_n) = lstm(packed)

# 4. 解压 (如果需要每个时间步输出)
out, lens = pad_packed_sequence(
    packed_out, batch_first=True)
```

为什么重要?

问题:

NLP中句子长度各不相同，必须用PAD填充到统一长度。但LSTM会“认真”处理PAD位置，浪费计算且引入噪声。

pack_padded_sequence 的作用

:

- 将填充后的tensor压缩成紧凑格式
- LSTM只处理实际存在的token
- 节省计算量，避免垂序列干扰梯度

性能对比:

平均句长20、最长100的batch:
使用pack后速度提升 2-3倍,
内存占用减少 40%+

结束

谢谢